INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

INGENIERÍA DE SOFTWARE

TALLER — COMPLEJIDAD ALGORÍTMICA Y ARREGLOS

SD1004 · Estructura de Datos

Estudiante: Anderson Echeverri Castrillon

Docente: Juan Duque

Semestre: 2026-II

Medellín, Colombia · 2026

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

1. Buscar una palabra en un diccionario físico, abriendo siempre por la mitad de las páginas que quedan.

Respuesta: O(log n)
Justificación: Cada vez se reduce aproximadamente a la mitad la cantidad de páginas que se deben revisar. Por eso la cantidad de búsquedas crece lentamente aunque aumente el número de páginas.

2. Revisar, uno por uno, cada carné en una caja de carnés de estudiantes hasta encontrar el que buscas.

Respuesta: O(n)
Justificación: En el peor caso es necesario revisar todos los carnés uno por uno. Si hay el doble de carnés, se puede necesitar revisar el doble de elementos.

3. Sacar la primera carta de un mazo ya barajado.

Respuesta: O(1)
Justificación: No importa cuántas cartas tenga el mazo; sacar la primera carta es una operación directa.

4. Comparar cada estudiante de un salón con cada uno de los demás estudiantes, para ver si algún par cumple años el mismo día.

Respuesta: $O(n^2)$
Justificación: Cada estudiante se compara con muchos otros. Al aumentar la cantidad de estudiantes, aumenta mucho la cantidad de comparaciones entre pares.

5. Consultar la hora en tu reloj.

Respuesta: O(1)
Justificación: Es una operación directa y no depende de la cantidad de otros datos.

1.2 De la vida real al análisis

Una actividad cotidiana que se puede considerar O(n) es revisar una lista de compras antes de salir del supermercado. Si tengo 10 productos, reviso aproximadamente 10 elementos; si tengo 20, debo revisar aproximadamente 20. Si la entrada se hace 10 veces más grande, el tiempo aumentaría aproximadamente 10 veces.

1.3 ¿Cuál escalar mejor?

Recomendaría la Forma B: O(n log n). Aunque para n = 5 la Forma A puede ser más rápida, O(n log n) crece más lentamente cuando aumenta la cantidad de datos. Si en el futuro hay miles o millones de datos, será mucho más conveniente que $O(n^2)$.

1.4 Verdadero o falso

a. Un algoritmo O(1) siempre es más rápido en segundos reales que uno O(n).

Falso. Big O no indica exactamente cuántos segundos tarda un algoritmo; indica cómo crece el trabajo al aumentar los datos.

b. La notación Big O describe cómo crece el tiempo de ejecución a medida que crecen los datos, no el tiempo exacto en segundos.

Verdadero. Big O permite analizar cómo aumenta el trabajo cuando aumenta la cantidad de datos.

c. Buscar por índice en un arreglo (arreglo[5]) es O(n), porque hay que recorrer los primeros 5 elementos.

Falso. El acceso por índice es directo y no necesita recorrer las posiciones anteriores; por eso es O(1).

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

1. Arreglo con sus índices:

```
Índice: 0 1 2 3 4 5
+-----+-----+-----+-----+-----+-----+
notas: | 3.5 | 4.2 | 2.8 | 5.0 | 3.9 | 4.5 |
+-----+-----+-----+-----+-----+-----+
```

2. La tercera nota ingresada es 2.8 y tiene índice 2. Los arreglos empiezan a contar desde 0.

3. Para acceder directamente a la última nota: notas[5]. De forma general: notas[tamaño - 1].

2.2 Direcciones de memoria

Datos: dirección base = 0x2000; cada valor = 4 bytes.

```
notas[0] = 0x2000 + (0 × 4) = 0x2000
notas[3] = 0x2000 + (3 × 4) = 0x200C
notas[5] = 0x2000 + (5 × 4) = 0x2014
```

Acceder a notas[3] es igual de rápido que acceder a notas[0] porque el computador puede calcular directamente la dirección usando la base, el índice y el tamaño de cada elemento. No necesita recorrer las posiciones anteriores. Por eso es O(1).

2.3 Búsqueda lineal vs. binaria

1. Búsqueda lineal para encontrar 31:

```
15 → No
18 → No
20 → No
23 → No
27 → No
31 → Sí
Cantidad de comparaciones: 6.
```

2. Búsqueda binaria:

```
Se revisa 23 → 31 es mayor, se descarta la mitad izquierda.
Luego se revisa 31 → encontrado.
Cantidad de comparaciones: 2.
```

3. La búsqueda binaria necesita que el arreglo esté ordenado porque decide qué mitad descartar comparando el valor buscado con el elemento del medio. Sin orden no se puede saber qué parte

eliminar.

4. Con 1 millón de elementos seguiría usando búsqueda binaria, siempre que el arreglo esté ordenado. La lineal puede revisar hasta aproximadamente un millón de elementos, mientras la binaria elimina la mitad en cada paso. O(log n) crece mucho más lentamente que O(n).

2.4 Insertar un elemento

1. Insertar al final con espacio disponible es O(1), porque solo se coloca el elemento en la siguiente posición. Si no hay espacio y hay que crear un arreglo más grande, puede ser O(n), porque hay que copiar los elementos.

2. Insertar 21 en la mitad para mantener el orden requiere desplazar los elementos que están a la derecha para abrir espacio: [15, 18, 20, 21, 23, 27]. Puede ser necesario mover varios elementos, por lo que es O(n).

3. Comparación:

Situación	Complejidad
Insertar al final con espacio	O(1)
Insertar al final sin espacio	O(n)
Insertar en la mitad	O(n)

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

1. Matriz de 3 filas × 4 columnas:

```
COLUMNAS
0 1 2 3
+-------+-------+-------+-------+
Fila 0 | [0][0]| [0][1]| [0][2]| [0][3]|
+-------+-------+-------+-------+
Fila 1 | [1][0]| [1][1]| [1][2]| [1][3]|
+-------+-------+-------+-------+
Fila 2 | [2][0]| [2][1]| [2][2]| [2][3]|
+-------+-------+-------+-------+
```

2. Para acceder al puesto de la fila 2, columna 3: salon[2][3].

3. La matriz tiene 3 × 4 = 12 casillas. En general: filas × columnas.

3.2 De 2D a memoria (row-major)

1. Aplanada en memoria, fila por fila:

```
[0][0] [0][1] [0][2] [0][3] [1][0] [1][1] [1][2] [1][3] [2][0] [2][1] [2][2] [2][3]
```

2. Dirección de [1][2]:

```
0x1000 + ((1 × 4 + 2) × 4)
= 0x1000 + (6 × 4)
= 0x1000 + 24
= 0x1018
```

Dirección de [1][2]: 0x1018.

3. Dirección de [0][0]: 0x1000.

```
Dirección de [2][3] = 0x1000 + ((2 × 4 + 3) × 4)
= 0x1000 + 44
= 0x102C
```

Dirección de [2][3]: 0x102C.

3.3 Recorrido con ciclos anidados

1. Pseudocódigo:

```
ocupados = 0

PARA fila DESDE 0 HASTA f - 1
PARA columna DESDE 0 HASTA c - 1
SI salon[fila][columna] == 1
ocupados = ocupados + 1
FIN SI
FIN PARA
FIN PARA

MOSTRAR ocupados
```

2. La complejidad es O(f × c), porque el primer ciclo recorre f filas y, por cada fila, el segundo recorre c columnas. Si la matriz es cuadrada y tiene n filas y n columnas, se expresa como $O(n^2)$.

3. Si f y c se duplican, (2f) × (2c) = 4fc. Por lo tanto, se recorren cuatro veces más casillas y el tiempo sería aproximadamente cuatro veces mayor.

3.4 Conectando todo

Big O nos ayuda a entender cómo aumenta el trabajo de un algoritmo cuando aumenta la cantidad de datos. Los arreglos 1D permiten almacenar elementos en una sola dimensión y acceder directamente a ellos mediante un índice. Los arreglos 2D permiten organizar la información usando filas y columnas, como una matriz. Dependiendo del problema, podemos elegir un arreglo 1D o 2D para organizar mejor los datos. También debemos pensar en la forma de buscar información: la búsqueda lineal es sencilla pero puede ser más lenta con muchos datos, mientras que la búsqueda binaria es más rápida pero necesita que los datos estén ordenados. Por eso conocer Big O ayuda a elegir una solución más eficiente.

Conclusión

En este taller se pudo comprender que la complejidad algorítmica permite analizar cómo cambia el rendimiento de un algoritmo cuando aumenta la cantidad de datos. También se reforzó el funcionamiento de los arreglos 1D y 2D, incluyendo sus índices, acceso a memoria, búsquedas e inserciones. Finalmente, se pudo observar que elegir correctamente una estructura de datos y un método de búsqueda puede hacer que un programa sea mucho más eficiente cuando trabaja con grandes cantidades de información.